2.12 Функции

1. Что такое функция?

Функция — это блок кода, выполняющий определенную задачу. Функции позволяют повторно использовать код и упрощают его поддержку.

2. Объявление и вызов функций

```
package main

func add(a int, b int) int {
    return a + b
}

func main() {
    result := add(2, 3)
    println(result)
}
```

3. Множественные возвращаемые значения

```
package main

func divide(a, b int) (int, int) {
    return a / b, a % b
}

func main() {
    q, r := divide(10, 3)
    println(q, r)
}
```

4. Именованные возвращаемые значения

```
package main

func swap(a, b int) (x int, y int) {
    x, y = b, a
    return
}

func main() {
    a, b := swap(1, 2)
    println(a, b)
}
```

MERION

5. Анонимные функции и замыкания

```
package main

 func main() {

    sum := func(a, b int) int {
    return a + b
    }

    result := sum(3, 4)
    println(result)
}
```

```
package main

func counter() func() int {
    count := 0
    return func() int {
        count++
        return count
    }
}

func main() {
    c := counter()
    println(c())
    println(c())
}
```

6. Variadic функции

Функции могут принимать переменное количество аргументов

```
package main

func sum(numbers ...int) int {
    total := 0
    for _, num := range numbers {
        total += num
    }
    return total
}

func main() {
    result := sum(1, 2, 3, 4)
    println(result)
}
```

MERION

В Go невозможно использовать **variadic** параметр не последним в списке параметров функции. Компилятор не позволит создать такую функцию.

```
func invalidFunction(numbers ...int, name string)
    { fmt.Println(numbers, name)
}
```

Variadic параметр принимает неопределенное количество аргументов, что мешает определить, какие параметры относятся к `numbers` , а какие — к `name`

```
func validFunction(numbers []int, name
    string) { fmt.Println(numbers, name)
}

func validFunction(name string, numbers
    ...int) { fmt.Println(numbers, name)
}
```

```go
package main

import "fmt"

func validFunction(name string, numbers ...int) {
    fmt.Println(numbers, name)
}

func main() {
    validFunction("hi there", []int{1, 2, 3, 5, 6}...)
}
```

7. Рекурсия

```
package main

func factorial(n int) int {
    print(n)
    print(" " )
    if n == 0 {
        return 1
    }
    return n * factorial(n-1)
}

func main() {
    print(factorial(5))
}
```

Итоги по теме

- Разобрали объявление функций, множественные возвращаемые значения и замыкания.
- Изучили variadic функции и рекурсию.
- Выполнили практические задания для закрепления темы.